

FSOC-PAT: AI-assisted virtual camera tracking

Technical report and presentation reference | 29 September 2026

1. Executive overview and scope

FSOC-PAT is a software testbed for coarse pointing, acquisition and tracking of a moving optical beacon. It renders a configurable scene, observes it through a constrained virtual camera, extracts target measurements, estimates motion and commands the camera to retain the target in its field of view. The implementation is an image-driven feedback loop: the detector observes the rendered camera image, while simulation ground truth is reserved for evaluation.

The reference is the user-supplied 26169.pdf, titled Development of an AI-Based Virtual Camera Tracking System for Coarse Alignment of Mobile Free Space Optical Communication Terminals. The statement identifies coarse alignment as the stage that locates a remote terminal and maintains visibility before fine alignment. This application demonstrates that software stage. It does not transmit a laser, establish a communications link or demonstrate fine beam steering on physical hardware.

Three detection approaches share the tracking and control infrastructure: CV uses classical candidate extraction and a calibrated feature score; AI scans the image using a small convolutional network; Hybrid applies that network to classical candidates. Hybrid is the featured design because it combines selective candidate generation with learned recognition. Its superiority remains a hypothesis to evaluate by scenario and metric, not an output to preconfigure.

The interface provides six entry points: Quick demo, Custom mission, Benchmark, Load video, Camera lab and Model comparison. Camera/world views and engineering overlays explain the current state; session results and PDF/CSV exports retain the measured outcome. Comparison runs are executed sequentially and then replayed together, rather than running three real-time controllers concurrently.

This document describes the inspected code, stored checkpoint metadata and existing recorded comparisons. Historical experiments are identified as historical; they are not new validation runs. A green camera alignment badge is a local interaction result, not evidence that every requirement passed. Classic rendering remains the fallback; Optical space is an optional synthetic lens-response experiment.

The project is intended as an accessible algorithm-development and demonstration platform. Hardware certification, calibrated optical propagation, real-space imagery validation and a verified standalone release remain separate tasks.

2. Problem statement and requirement interpretation

The supplied statement requests a minimum 2000 x 2000 virtual screen, a 640 x 480 camera, default 4 x 3 degree field of view, at least 30 Hz camera update and at least 20 Hz motion updates. It requests a beacon target, one mandatory target, default square shape, default 10 x 10 pixel size with a suggested 5-20 pixel range and a user-defined initial location. Required motion choices are straight, circular, figure-eight and random; spiral and sinusoidal are optional.

Performance targets are acquisition ≤ 2 s, tracking error ≤ 10 px, target loss $< 5\%$, reacquisition ≤ 1 s and processing speed ≥ 20 FPS. The application previously used $\leq 5\%$ for loss; this revision changes new evaluations to the statement's strict $< 5\%$ boundary. A result of exactly 5% must fail that check. Other metrics retain their existing definitions; these are explained in Section 8.

The phrase tracking error does not specify a complete aggregation protocol. The application checks mean pointing error in simulation and centroid error in scored video. It also records maximum error, RMSE and ground-truth centroid statistics. A mean below 10 px does not establish that every frame remained below 10 px. The evaluator should agree on aggregation, exclusions and timing before treating an application PASS as full compliance.

The statement explicitly includes Clear, Haze, Fog, Rain and Low light, plus salt-and-pepper, Gaussian and Poisson noise, camera jitter up to ± 20 px/frame and platform motion. Atmospheric weather belongs to a terrestrial or atmosphere-crossing link scenario. Vacuum inter-satellite operation does not contain terrestrial rain or fog along the vacuum path. A star-themed background is interface decoration and does not change this distinction.

Mandatory deliverables include a standalone executable, documented source, a technical report of about 10-15 pages, a user manual and automatic performance logs. A 3-5 minute video is optional. Running the local Python server in a browser is not proof that standalone packaging is complete. The supplied evaluation allocates 20% to functional verification, 30% to scenario benchmarks, 30% to recorded-video benchmarks and 20% to technical evaluation.

Traceability anchors: config/default.yaml; evaluation/requirements.py; simulation/camera.py; disturbance/disturbance.py; input/video_source.py; evaluation/gt_metrics.py.

3. Architecture and data flow

The browser interface is plain HTML, CSS and JavaScript, with Chart.js plots. It is not a React application. FastAPI serves the interface and exposes commands, session records, video preparation, comparison jobs and live streams. Pygame renders simulation frames; OpenCV handles image operations; PyTorch runs the classifier; NumPy supports numerical calculations; ReportLab produces PDFs.

Live flow: configuration -> scene and target update -> camera rendering -> image disturbances -> detection/scoring -> tracking and state fusion -> camera servo -> frame telemetry and evaluation -> browser views and persistent exports. The camera command affects subsequent observations, closing the feedback loop. Camera latency and slew limits matter because the controller cannot simply teleport the viewport to the target.

SimulationThread in ui/sim_thread.py orchestrates the pipeline. A frame packet pairs the clean camera RGB image with its telemetry. The browser associates overlays with the displayed frame, reducing the risk of a box drawn using a newer state over an older image. Ground-truth fields support scoring and diagnostics, while tracking estimates and accepted detections supply the operational overlay.

Session finalization is a separate lifecycle: stop execution, compute authoritative summary, export artifacts and publish the session manifest. Atomic manifest replacement reduces partial-write exposure. Failures must remain visible and must not be converted into empty success reports. Session identity links the summary, per-frame CSV, PDF, configuration and events.

Comparison flow is isolated: a fixed scenario is serialized, one child process runs each mode, each child stores telemetry and preview frames, and the browser synchronizes playback on source time. Independent cameras see different image crops as their control loops evolve. The target schedule and exogenous random schedule are shared; identical video pixels are not promised for this closed-loop comparison.

Key interfaces are server/app.py, server/mission_config.py, server/sessions.py, server/video_preflight.py and server/comparison.py. Core calculation modules should remain independent of decorative home-page changes. The homepage SVG and card styling do not enter sensor images or require retraining.

Engineering principle: visual polish explains evidence; it does not create evidence. Every measured value should be traceable to a saved frame or reduction rule.



Closed loop: the camera command changes the next observation; ground truth is used for scoring.

4. Virtual scene, camera geometry and disturbances

Default configuration uses a 2000 x 2000 world with 800 background stars, a 640 x 480 camera and one bright square beacon. Default field of view is 4 degrees horizontally and 3 vertically. Both axes therefore have 160 pixels per degree in the configured approximation. Ten sensor pixels correspond to 0.0625 degrees at this default scale; this is a simulator conversion, not a calibrated physical-camera claim.

The virtual camera maps world positions into image coordinates around its current centre. Pan changes horizontal pointing, while positive tilt moves pointing upward and uses the image's downward-positive vertical convention. Limits and speed caps constrain commands. The default maximum pan and tilt speed is 5 degrees/s and the configured latency is two frames. A nominal 60 Hz simulation step does not guarantee the computer sustains 60 processed frames/s.

Target trajectories include straight, circular, figure-eight, random, spiral and sinusoidal paths. Runtime motion transitions preserve the current position with a short velocity transition. This avoids resetting the target to the world centre whenever the motion selection changes. Seeded random motion supports repeatable experiments; switching motion is an event that belongs in the session record.

DisturbanceEngine modifies the camera image with noise, turbulence, vibration, scintillation, transient motion and atmospheric appearance. Haze reduces contrast; fog adds stronger degradation and blur; rain introduces moving streaks with dimming; low light changes gain and sensor noise. These are controlled image-space stressors, not a validated weather or radiative-transfer simulator. Platform perturbation likewise needs to be distinguished from a full vehicle dynamics model.

Classic and Optical space are separate render profiles. OpticalWorld first renders the classic scene, then applies a small Gaussian point-spread response and restrained highlight halo before detection. The model receives the modified pixels. It is therefore a sensor-domain change and must be evaluated for detection, tracking and throughput. The profile is synthetic, not photorealistic or validated against space-camera data. No new training is performed when the user selects it.

Useful future refinements include documented camera exposure, point-spread calibration, saturation and shot-noise parameterization, and separate vacuum/atmospheric scenario presets. These should be versioned and tested against the classic baseline before replacing defaults.

5. CV, AI and Hybrid detection

CV: BeaconDetector converts the camera frame, cleans noise, extracts bright candidate structures and estimates candidate centre, radius, brightness and shape properties. CVScorer computes a calibrated score from hand-designed features. The configured feature score uses fitted weights; it is more precise to call this a classical feature pipeline with a calibrated scorer than an entirely untrained algorithm. It does not run BeaconCNN at inference.

AI: AIScanner applies the same BeaconCNN used by Hybrid to a whole-frame confidence map. The nominal patch is 32 x 32 pixels at stride 8. Shared convolutions avoid separately recomputing the feature extractor for every patch. Local maxima, non-maximum suppression and local intensity-centroid refinement produce candidates. At 640 x 480, the map contains approximately $77 \times 57 = 4389$ candidate window positions before peak selection. Boundary context differs from individually zero-padded cutout classification, so exact equivalence should not be claimed.

Hybrid: the classical detector proposes a relatively small set of regions; the CNN scores their 32 x 32 grayscale patches in a batch. This targets learned classification work at plausible beacons. Hybrid's intended advantage is a useful balance of recognition and compute cost. A missed classical proposal remains a limitation: the classifier cannot recover a beacon it never receives. Full-frame AI may help in cases where proposal extraction fails, while simple CV can be competitive on clean scenes.

All three approaches share the motion estimator, confidence history, search logic and camera control. Their similar traces on a clean single-beacon scene are plausible, not proof of fake data. The common controller may dominate the pointing dynamics once each mode provides similar centroids. To demonstrate differences, report missed detections, false locks, coverage and processing cost in addition to pointing RMSE.

Presentation convention: CV is amber dashed, AI purple dotted and Hybrid teal bold solid. Hybrid is visually featured. Per-metric leaders are derived from measured values with ties preserved at displayed precision. Lower error and loss are better; higher correct-lock percentage and processing rate are better. No mode is forced to win. A lower error from very few valid samples must not outrank broad reliable tracking without explaining coverage.

Source files: `detection/detector.py`, `detection/cv_scorer.py`, `detection/ai_scanner.py` and `detection/classifier.py`.

6. CNN architecture, training and domain limitations

BeaconCNN accepts one 32 x 32 grayscale patch normalized to [0,1]. The feature extractor contains three 3 x 3 convolution blocks with 8, 16 and 32 output channels. Each block uses BatchNorm, ReLU and 2 x 2 max pooling. The feature tensor becomes 32 x 4 x 4, flattened to 512 values. The classification head is 512 -> 64 -> 2 with ReLU and dropout 0.4 during training. Softmax yields distractor/beacon scores at inference.

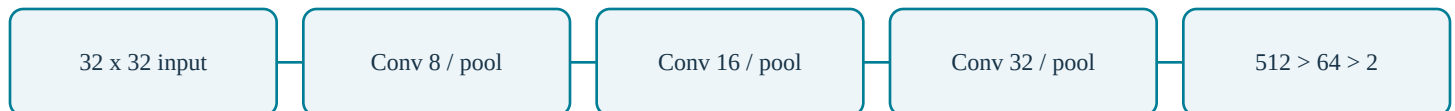
Direct inspection counts 38,962 trainable parameters. The current default checkpoint is models/beacon_classifier_v2.pt. Its stored metadata reports epoch 23, training accuracy 0.9988877 and validation accuracy 0.9994598, or about 99.946%. These are stored training-run statistics, not a fresh reevaluation, test-set result, real-world accuracy or tracking-success percentage. The older comment claiming roughly 25,000 parameters should not be repeated in presentations.

The repository training script specifies batch size 64, 25 epochs, learning rate 0.001, weight decay 0.0001 and a step learning-rate schedule. Augmentations include flips, 90-degree rotation, brightness changes and small Gaussian noise. Inference runs on CPU with the network in evaluation mode. CPU performance depends on thread settings, workload, image processing and candidate count.

The v2 data generator renders simulator-derived examples under varied appearance conditions and mixes earlier synthetic patches into its training/validation pool. It uses training scene seeds 1000-1005 and separate test seeds 2000-2001. The train/validation split is a shuffled patch split; it does not establish scene-independent generalization within that pool. Harness seeds are separate, but visual-domain similarity remains. Synthetic seed separation is useful and still weaker than held-out recordings from physical cameras.

Changing only the home-page background has no model impact. Changing sensor-rendered stars, bloom, blur or beacon shape may shift the model input distribution. First evaluate the unchanged model on the new profile across noise, motion and visibility conditions. Retrain only if evidence warrants it, preserve the classic model and renderer, and keep a held-out test set. A high-quality looking image does not imply better detector accuracy.

No weights, dataset splits or training settings were changed in this reporting and presentation revision. Checkpoint metadata should be accompanied by an exported evaluation protocol before appearing as an accuracy claim on a title slide.



Each convolution block includes BatchNorm and ReLU. Output: distractor / beacon scores.

7. Tracking, control and engineering visibility

MasterTracker combines motion estimation, temporal confidence fusion, image stabilization and search. The default prediction model is an interacting multiple-model tracker combining constant-velocity and coordinated-turn behaviour. A single constant-velocity Kalman alternative is also available. Prediction supports short measurement gaps; it does not turn an unobserved beacon into a measured detection.

The state progression is SEARCHING -> ACQUIRING -> LOCKED -> COASTING -> LOST -> SEARCHING, with reacquisition able to restore LOCKED. Temporal evidence helps avoid declaring a lock from one bright distractor. SceneEstimator derives degradation cues from the observed image rather than reading the simulator's disturbance settings. ImageStabiliser estimates background motion; elongated bright streak handling reduces false stabilization from simulated rain.

CameraServo uses smoothed velocity feedforward and a proportional centring correction applied relative to the commanded camera target. Its locked-loop correction gain is 0.15 with a 2 px dead zone. This structure accounts for commands already moving through latency rather than repeatedly correcting against a stale actual position. The repository also contains PID support; it would be inaccurate to describe the entire current locked-loop controller simply as a conventional PID.

The camera view always shows a fixed-size tracking marker when a usable position exists. A green box indicates a currently measured locked target; a dashed amber box indicates prediction. This box marks the tracker position, not an accurately segmented object extent. Engineering mode adds the accepted measurement, next-step estimate, candidate markers and scores, a centre-error vector and frame processing time. Cyan denotes measurement/error annotations and violet the prediction. Colour is reinforced by labels and line styles.

Camera Lab fixes the world beacon and lets the user pan and tilt the camera. Manual and assist modes preserve user control; assist adds direction hints. Auto mode controls the camera. Alignment requires a measured LOCKED position within 10 px of centre for 0.5 source seconds, with a 14 px exit threshold to reduce flicker. Lost measurements clear alignment; stale feed detection clears the badge. Green means aligned under this interaction rule, not PS certification.

The engineering view makes the feedback loop inspectable. Avoid adding decorative uncertainty ellipses or future paths unless supported by the estimator's actual outputs and a clear coordinate convention.

8. Metrics, timing and honest pass/fail interpretation

Centroid error is the Euclidean distance between estimated target position and known target position in the image. Pointing error is the distance from the estimated target to image centre, preferably after stabilization for the simulator's pointing measure. These answer different questions: a target can be detected accurately while remaining off-centre. For a prerecorded video, the app cannot steer the recording, so scored video accuracy is centroid error rather than camera-centre distance.

For valid errors e_i , $\text{mean} = \sum(e_i)/N$, $\text{maximum} = \max(e_i)$, and $\text{RMSE} = \sqrt{\sum(e_i^2)/N}$. Missing samples are not zero. The scoring module considers a lock correct when state is LOCKED and the estimate is within 25 px of ground truth. That association tolerance is separate from the 10 px pointing target. Pointing summaries exclude each lock stretch's initial pull-in until error is below 10 px or one source second has elapsed. These exclusions must accompany performance claims.

Target loss is evaluated after the first correct lock: the number of frames in which the beacon is in FOV but not correctly locked divided by all frames after that first lock. Out-of-FOV percentage is reported separately. Consequently, a low loss percentage alone can hide a poor camera that loses the beacon out of frame; assess both. Correct-lock/all-frames, visible-frame lock and tracker-state lock also use different denominators and should retain their labels.

Acquisition with ground truth measures first in-FOV observation to first correct lock. Without GT, the app can only report its own detection-to-lock timing. Reacquisition uses explicit availability/event rules; time while the beacon is unavailable may be excluded. No observed recovery event means NOT TESTED, and incomplete recovery can fail. A reported zero can result from lock being restored before the defined recovery clock starts; it does not imply zero compute latency.

Wall time, source time, pipeline-only FPS, full-step throughput and browser playback FPS must remain distinct. Comparison uses a fixed 1/60 s source step, but generating five source seconds can take longer than five wall seconds. Its end-to-end realtime check is NOT TESTED. Pipeline FPS excludes some rendering/decoding work and must not be substituted for that check.

PASS requires every evaluated check to pass. FAIL records a failed check; INCOMPLETE retains untested checks; video without GT is NOT SCORED for accuracy. Historical exports keep their original scoring policy. New sessions use scoring version 3-ps-strict-loss; historical numbers are not silently rewritten.

9. Comparison methodology and recorded performance

The beta comparison generator supports straight, figure-eight and sinusoidal motion, a fixed seed of 42 and optional common 0.5 s observation dropout. Default mild sensor noise and independent mode processes provide a reproducible software comparison. Each source second contains 60 steps; five seconds means 300 steps per mode and 900 total. Preview images are stored every five steps plus the final frame, while telemetry remains at every source step.

AI generation is often slower because it computes whole-image CNN features and scores thousands of window locations. Hybrid usually classifies far fewer proposals. Current generation runs the modes sequentially with one Torch and OpenCV thread per worker. Preview encoding and storage add overhead. An identical completed run may be reused based on scenario, code and model fingerprints; its timings remain historical. Force fresh run requests new timing measurements.

The following table is generated from every complete comparison currently saved directly under logs/comparisons. It is a descriptive inventory, not a balanced multi-seed experiment or a new benchmark. It deliberately retains failures and ties. Error values may be conditioned on valid locks; review sample counts and loss before interpreting a small RMSE.

Run / motion / seconds	Mode	RMSE px	Lock %	Loss %	Pipeline FPS
26e5c2d3 / straight / 5.0	CV	1.694	87.667	10.239	32.024
26e5c2d3 / straight / 5.0	AI	1.666	87.667	10.239	1.567
26e5c2d3 / straight / 5.0	HYBRID	1.694	87.667	10.239	28.294
8143c19e / straight / 5.0	CV	1.799	97.667	0.000	29.129
8143c19e / straight / 5.0	AI	1.823	97.667	0.000	1.637
8143c19e / straight / 5.0	HYBRID	1.799	97.667	0.000	28.924
cde09336 / figure8 / 10.0	CV	4.043	93.833	5.059	27.910
cde09336 / figure8 / 10.0	AI	4.063	93.833	5.059	1.650
cde09336 / figure8 / 10.0	HYBRID	4.043	93.833	5.059	25.221

The stored runs do not establish universal Hybrid dominance. Clean-scene CV and Hybrid may provide nearly identical positions, and timing changes with machine load. To support a stronger claim, define a representative scenario matrix before running it, repeat across seeds, include clear and degraded scenes, preserve failed runs, summarize distributions and compare accuracy together with speed and coverage.

For presentation, say: Hybrid combines classical localization with learned candidate verification. Then show the measured comparison, naming which metrics improve and which are tied or worse. Do not claim that a visually bolder curve means higher accuracy. The beta chart gives Hybrid a bold solid teal trace for legibility while preserving all original values.

10. User workflows, reports and source map

Quick demo starts a prepared scenario. Custom mission exposes motion, speed, initial position, environment and detector selection, then provides a mission brief before execution. During execution, camera and world views show acquisition and pointing. Stop finalizes one session and opens results. The homepage recent list and Reports archive reopen saved sessions by identity.

Load video prepares an MP4, displays metadata and thumbnails and optionally accepts a ground-truth CSV. The coordinate convention and frame numbering must be selected explicitly; the application canonicalizes GT to zero-based processing coordinates at 640 x 480. Without GT, do not interpret confidence or state-lock percentage as accuracy. The statement's video benchmark bypasses the virtual PTZ camera; recorded input cannot be physically recentered by the controller.

Benchmark runs configured scenarios and reports finalized backend measurements. A short preview run is a preview, not a replacement for the intended benchmark duration. Model comparison is for synchronized replay and relative inspection; its unpaced timing limits remain visible. Camera Lab is an interactive explanation of pointing ownership and acquisition rather than a separate certification workflow.

Source map: simulation/ contains world, target, motion, camera and optical rendering. disturbance/ contains image perturbations. detection/ contains candidates, scoring, CNN, scanner and training scripts. tracking/ contains prediction, temporal fusion, stabilization and search. control/ contains servo and PID support. ui/sim_thread.py integrates the runtime. input/ handles video input. comparison/ contains scenario and worker generation. evaluation/ defines metrics, requirements, manifests and reports. server/ exposes APIs; frontend/src/ holds the browser interface; tests/ and harness/ hold automated checks and experiment infrastructure.

Each session should retain summary CSV, frame CSV, session JSON and PDF. New PDF exports share table treatment and footer styling. Comparison tables and the browser leader panel share evaluation/comparison_presentation.py. Print colours are darker equivalents of the screen palette for white-paper contrast. Existing PDF files are historical artifacts, not automatically rewritten by a UI change.

Run the backend from F:\FSOC with venv\Scripts\python.exe -m server.app, then open <http://localhost:8765>. Use one backend instance. If the port is occupied, identify the existing project process before restarting it. The current browser route avoids dependence on an Electron launcher resolving the wrong executable. A fresh Python start is required after backend changes; use a hard browser refresh for updated assets.

11. Verification, risks and next engineering work

Verification should cover algorithms, data contracts, lifecycle and presentation. Unit tests cover motion continuity, session records, finalization, Camera Lab, comparison contracts and no-leak expectations. This revision adds explicit checks that Hybrid emphasis cannot force a win, ties and missing values remain visible, and exactly 5% target loss fails. Browser checks exercise actual routes, chart rendering, mobile layout and keyboard interaction. PDF QA must inspect rendered pages, not merely successful file creation.

Prior handoff evidence records live workflow tests and a separate behavioural no-leak test. Those are historical results, not a claim that every scenario was rerun today. The current revision's verification summary is recorded separately in handoff/REVIEW_V6.md. No CNN retraining, complete benchmark matrix or hardware test is implied by passing software tests.

Main limitations: image-space geometry and disturbances are approximations; classifier validation is synthetic; summary errors use conditional sample sets; elapsed processing rate depends on the machine; historical sessions may use older policies; weather presets are not vacuum physics; fixed marker extent is not object segmentation. The single-target architecture should not be described as validated multi-object identity tracking merely because distractors can be rendered.

Priority 1 is metric agreement: document the evaluator's error aggregation, recovery interval, loss denominator and end-to-end FPS measurement. Priority 2 is a repeatable multi-seed matrix, with untouched holdout data and full failure retention. Priority 3 is actual recorded-camera validation, including subpixel labelled centroids and timestamp alignment. Priority 4 is a tested standalone Windows package with bundled assets, model files and a concise user manual. Packaging should be tested on a clean machine.

Longer-term extensions include calibrated optical rendering, exposure and saturation models, controlled hardware-in-the-loop tests, explicit atmospheric versus vacuum scenario labels and physically meaningful angular error. Any comparison optimization should preserve source frames and algorithms; compute shortcuts need equivalence tests or separate labels.

For a credible demonstration, show an easy acquisition, a controlled disturbance, a recovery, the saved evidence and one limitation. A transparent failure explanation is stronger than a fabricated PASS. Keep the classic renderer and existing checkpoint as baselines while experimental features remain optional.

12. Presentation narrative, references and delivery checklist

A suitable central claim is: FSOC-PAT demonstrates an image-driven coarse-alignment loop for a constrained virtual camera, with classical, learned and hybrid detection routes and reproducible performance evidence. This is specific enough to defend in questions and broad enough to explain the complete project.

Use a ten-slide narrative: (1) narrow-beam alignment problem; (2) exact supplied requirements; (3) closed-loop architecture; (4) virtual camera and disturbances; (5) three detection routes; (6) CNN and training limitations; (7) estimator and camera servo; (8) measured comparison with coverage and timing; (9) live demonstration and report; (10) limitations and future validation. Pair each numeric claim with its run ID, metric definition and timing basis.

For a four-minute video, devote roughly 25 s to the problem, 30 s to the architecture, 45 s to acquisition and engineering overlays, 40 s to Camera Lab, 45 s to recorded comparison, 35 s to exported evidence and 20 s to limits and next steps. The companion PRESENTATION_AND_DEMO.md contains speaker-ready narration, shot directions and questions to prepare for. Use pre-generated comparison playback so video duration is not dominated by computation.

Primary reference: the supplied 26169.pdf, pages 1-3. Page 1 defines the objective and most camera/target/performance parameters. Page 2 contains the atmospheric disturbance row, remaining performance criteria and deliverables. Page 3 continues evaluation criteria and identifies Department of Space / ISRO. The original PDF is preserved at C:/Users/shrir/Desktop/FSOC SIH related docs/26169.pdf.

Implementation references: config/default.yaml; ui/sim_thread.py; detection/classifier.py; detection/ai_scanner.py; detection/generate_training_data.py; tracking/tracker.py; control/camera_servo.py; evaluation/gt_metrics.py; evaluation/requirements.py; comparison/scenario.py; comparison/worker.py; evaluation/comparison_presentation.py. Repository comments are useful context but do not substitute for observed code and test results.

Evidence references: recorded result.json files under logs/comparisons; checkpoint metadata in models/beacon_classifier_v2.pt; historical validation in handoff/IMPLEMENTED.md and handoff/POLISH_V3.md; overlay verification in handoff/OVERLAYS_V4.md; this revision's checks in handoff/REVIEW_V6.md. Full model provenance should include hashes and an independently reproducible evaluation before release.

Before submission: agree on scoring definitions; complete the scenario matrix; test provided 30 FPS videos; validate executable packaging; include installation/configuration guidance; verify every report number against its saved source; record a 3-5 minute demonstration; and remove unsupported claims of real-space validation, universal Hybrid superiority or patch accuracy being tracking accuracy.